21. Complexity, Recursion, and Big-O Practice

21. Complexity, Recursion, and Big-O Practice

Phase: Phase 4 — Algorithms and data structures
Estimated time: 1–2 days
Slug: `21-complexity-recursion`

Learning objectives

- Measure time/space with Big-O
- Trace recursive call trees
- Convert recursion to iteration when needed
- Estimate cost before coding

Prerequisites

Complete prior numbered lessons. Type every example yourself — reading Go is not writing Go.

Why this matters

Complexity literacy stops you from shipping accidental O(n^2) loops in hot paths. Recursion and Big-O are tools for designing algorithms in Go.

This lesson is a building block for later modules in the Goodway path. If you only skim, the exercises will feel like busywork; if you type the examples and narrate them aloud, the same patterns will reappear in HTTP handlers, concurrency, and AI service glue.

Core concepts

- Count dominant operations as n grows.
- Recursion needs base cases and progress toward them.
- Memoization trades memory for time.
- Prefer iterative forms when stack depth worries you.
- Measure with benchmarks when constants matter.

Hold these in working memory while you code. Each bullet should map to something you can demonstrate with a three-line program or a failing compile error.

Worked Go examples

Create a scratch module and type this yourself (do not paste blindly):

```
mkdir -p /tmp/gw-complexity, && cd /tmp/gw-complexity,
go mod init example.com/scratch
```

Then create `main.go`:

```
package main
```

```
import "fmt"

func Fib(n int) int {
    if n < 2 {
        return n
    }
    a, b := 0, 1
    for i := 2; i <= n; i++ {
        a, b = b, a+b
    }
    return b
}

func main() { fmt.Println(Fib(10)) }
```

Run it:

```
go run .
```

Expected output:

```
55
```

If your output differs, diff character-by-character — especially whitespace and types. Then change one line deliberately and predict the new output before running.

Mini-lab (`go run` steps)

1. Implement iterative Fib; compare to naive recursive for n=40 timing.
2. Estimate complexity of nested loops in a small snippet.
3. Rewrite recursion as an explicit stack.

Write down one sentence explaining *why* the output matches the code before you move on. Time-box the lab to 15–25 minutes; struggle is part of encoding the skill.

Curriculum notes (expanded from the compact draft)

Big-O intuition

Big-O describes how runtime/memory grow as input size n grows. Common classes: O(1), O(log n), O(n), O(n log n), O(n^2), O(2^n).

```
// O(n)
func Sum(xs []int) int {
    s := 0
    for _, v := range xs { s += v }
    return s
}
```

Nested loops over the same n often mean O(n^2). Sorting good algorithms are O(n log n).

Recursion

```go
func Fib(n int) int {
    if n < 2 { return n }
    return Fib(n-1) + Fib(n-2) // exponential without memo
}
```

Always define base cases. Watch stack depth — Go stacks grow, but recursion depth still matters.

Practice

For every algorithm you write: state expected complexity, then verify with benchmarks on growing n.

Failure modes to watch while coding

- Calling naive Fib-like recursion in production
- Confusing average vs worst case
- Ignoring space complexity of maps/slices

When a test fails, ask which failure mode you hit before changing code at random.

Idioms vs Python pitfalls

- Same Big-O ideas; Go's recursion depth limits still apply.
- list append amortizes like Go append.
- Prefer clarity first, then complexity improvements with tests.

When you catch yourself reaching for a Python habit, pause and rewrite the Go way once on purpose — even if both would "work". The goal is a Go-shaped intuition.

Going deeper

Interviewers care that you can discuss trade-offs while writing correct Go.

Practice checklist

Before the exercise, confirm you can:

1. Explain "Complexity, Recursion, and Big-O Practice" aloud in under two minutes without notes.
2. Re-type the worked example from memory and match the expected output.
3. Name the Go standard library packages touched by this lesson.
4. List one production bug this lesson's rules prevent.
5. Contrast the Go approach with the Python instinct listed above.
6. Write a three-line cheat sheet: one idiom, one pitfall, one command to remember.

If any item fails, re-run the mini-lab once more, then attempt the exercise tests.

Transfer to production

Ask where this shows up outside toys: config loading, API handlers, data pipelines, CI scripts, or on-call debugging. Write one concrete two-sentence scenario tying the worked example to that scenario. Tomorrow, when you rewrite the exercise from memory, check whether the code still fits that framing.

Common mistakes (especially from Python instincts)

- Calling naive Fib-like recursion in production
- Confusing average vs worst case
- Ignoring space complexity of maps/slices

Exercises

Work in `exercises/21-complexity-recursion/`. Read that folder's `README.md` before you code.

What you will build

Implement Fib (or the drill's function) efficiently enough for tests.

Study loop

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/21-complexity-recursion/` until you have an honest failing attempt.

Self-check

Answer without scrolling up:

1. What does O(n log n) typically describe?
2. Why is naive Fib exponential?
3. How do you empirically check complexity?

Further reading

- https://en.wikipedia.org/wiki/Big_O_notation